LABORATORY 1 — FULL WALKTHROUGH & CHEATSHEET

React Native + Expo + NativeWind (Tailwind CSS) | Windows | Android Emulator

Use this sheet from setup through submission. The uploaded lab manual specifies Expo SDK 57, NativeWind 4.2.7 and Tailwind CSS 3.4. Commands below are for Windows Command Prompt unless noted. D: drive setup is supplemental and optional.

0. What you need

Computer with internet; Node.js (the lab recommends Current LTS v22.13 or newer); npm; Visual Studio Code; Android Studio; Android SDK; Android Virtual Device (emulator). Git is optional. The emulator is the recommended preview method for this SDK 57 lab.

Suggested project path: D:\Projects\lab1-hello-rn. If you prefer another folder, substitute it consistently. Keep your normal Windows user profile on C:; do not move the whole profile for this lab.

1. Optional D: drive setup

1A. Android Studio Start Menu folder is NOT the SDK folder

During Android Studio installation, the Start Menu folder only controls where the shortcut/group appears in the Windows Start Menu. Leaving the default is fine. It does not choose the Android SDK location. After installation, open Android Studio → More Actions/Settings → SDK Manager and set Android SDK Location to D:\Android\Sdk if you want the SDK on D:. Install a recent SDK platform (API 34 or newer per the lab), Android SDK Platform-Tools and an emulator system image.

```
mkdir D:\Android\Sdk
mkdir D:\Android\AVD
mkdir D:\Android\UserHome
mkdir D:\Android\Gradle
```

Use Android Studio's SDK Manager to set the actual SDK path. Then Windows Search → Edit environment variables for your account → add these user variables:

Variable	Value
ANDROID_HOME	D:\Android\Sdk
ANDROID_SDK_ROOT	D:\Android\Sdk
ANDROID_AVD_HOME	D:\Android\AVD
GRADLE_USER_HOME	D:\Android\Gradle

In the user Path variable, add: %ANDROID_HOME%\platform-tools. Save, close terminals, then open a new terminal.

1B. Optional command-line environment variable setup

```
setx ANDROID_HOME "D:\Android\Sdk"
setx ANDROID_SDK_ROOT "D:\Android\Sdk"
setx ANDROID_AVD_HOME "D:\Android\AVD"
setx GRADLE_USER_HOME "D:\Android\Gradle"
```

These setx commands do not add platform-tools to Path. Add %ANDROID_HOME%\platform-tools using the Environment Variables GUI. Do not use setx to rewrite your entire Path. The AVD/Gradle locations are optional; Android Studio and Gradle must use those locations for the space savings to apply.

1C. Optional npm download cache on D:

```
npm config set cache "D:\npm-cache" --global
npm config get cache
```

The second command should print D:\npm-cache. This moves npm's cache, not the project folder.

1D. Verify Node and Android tools

```
node -v
npm -v
adb --version
adb devices
```

Version numbers should appear for Node, npm and adb. Start the emulator and run adb devices again; an emulator should be listed. If adb is not recognized, check ANDROID_HOME and the Path entry, then reopen the terminal.

2. Install Android Studio and create emulator

1. Download Android Studio from https://developer.android.com/studio and choose Standard setup.

2. Open Android Studio and wait for SDK components to finish downloading.

3. Check SDK Manager → Android SDK Location. If using D:, set it to D:\Android\Sdk here. The installer Start Menu folder is unrelated.

4. Install a recent Android platform (API 34 or newer as the manual says), Platform-Tools and a system image.

5. Open Device Manager (welcome screen: More Actions → Virtual Device Manager).

6. Click Create Device, choose a phone such as Pixel 7, choose/download a recent system image, then Finish.

7. Click Play beside the device and wait for the Android home screen.

If the emulator fails to start, check BIOS virtualization (AMD-V on Ryzen systems) and enable Windows Hypervisor Platform if needed. Close heavy programs or try a lighter device.

3. Create the Expo project

3A. Open a terminal in your projects folder

```
mkdir D:\Projects
cd /d D:\Projects
```

3B. Generate the SDK 57 blank project

```
npx create-expo-app@latest lab1-hello-rn --template blank@sdk-57
```

Meaning: npx runs a package command; create-expo-app@latest is the project generator; lab1-hello-rn is the folder name; --template chooses the starter; blank@sdk-57 requests the blank SDK 57 template. If prompted, choose SDK 57 as the manual directs.

3C. Enter project and open VS Code

```
cd lab1-hello-rn
code .
```

If code . is not recognized, open VS Code → File → Open Folder → D:\Projects\lab1-hello-rn. If the project has App.tsx instead of App.js, use App.tsx in place of App.js throughout.

3D. Run it

```
npx expo start
```

Wait for Metro's shortcut menu. Start the emulator first, then press a in the terminal. First launch can take a few minutes. Ctrl+C stops the server. Optional: w opens web preview; physical phone use is optional and SDK 57 may not match the newest Expo Go.

4. Part C — Hello World

FILE: App.js — replace its entire contents with:

```
import { View, Text } from 'react-native';

export default function App() {
  return (
    <View
      style={{
        flex: 1,
        alignItems: 'center',
        justifyContent: 'center',
      }}
    >
      <Text>Hello, World!</Text>
    </View>
  );
}
```

Line guide: import makes View and Text available from react-native. export default makes App the file's main export. return describes the interface in JSX. View is a layout container; Text displays text. flex: 1 fills available parent space. alignItems centers children across the cross-axis (horizontal in a column); justifyContent centers along the main axis (vertical in a column).

Do not put plain text directly inside View. Use <Text>Hello, World!</Text>, or React Native reports that text strings must be rendered within a Text component.

5. Part D — Install and configure NativeWind

Stop Expo with Ctrl+C. Run each command separately, from inside lab1-hello-rn. Keep each command on one line.

```
npm install nativewind@4.2.7
```

```
npm install --save-dev tailwindcss@^3.4.17 prettier-plugin-tailwindcss@^0.5.11 babel-preset-expo
```

```
npx expo install react-native-reanimated react-native-worklets react-native-safe-area-context
```

```
npx expo install --check
```

NativeWind provides Tailwind-style classes for React Native. The lab uses NativeWind v4 with Tailwind CSS v3. Expo install selects compatible versions of native-related packages. Do not run npm audit fix --force just because warnings appear; forced upgrades can break compatibility.

5A. FILE: tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: ['./App.{js,jsx,ts,tsx}', './components/**/*.{js,jsx,ts,tsx}'],
  presets: [require('nativewind/preset')],
  theme: {
    extend: {},
  },
  plugins: [],
};
```

content lists files Tailwind scans for classes. presets loads NativeWind's preset. theme.extend is for custom design values and remains empty here. plugins is empty. If you add components elsewhere, include their paths in content.

5B. FILE: global.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Create this file in the project root beside App.js. The directives include Tailwind base styles, component-layer styles and utility classes.

5C. FILE: babel.config.js

```
module.exports = function (api) {
  api.cache(true);
  return {
    presets: [
      ['babel-preset-expo', { jsxImportSource: 'nativewind' }],
      'nativewind/babel',
    ],
  };
};
```

Babel transforms JavaScript and JSX. api.cache(true) allows configuration caching. The Expo preset and NativeWind integration must match the lab setup.

5D. FILE: metro.config.js

```
const { getDefaultConfig } = require('expo/metro-config');
const { withNativeWind } = require('nativewind/metro');

const config = getDefaultConfig(__dirname);

module.exports = withNativeWind(config, {
  input: './global.css',
});
```

Metro bundles the app. getDefaultConfig loads Expo's default configuration; withNativeWind integrates NativeWind; input points to global.css.

5E. FILE: app.json

Keep all existing settings. Ensure the expo object contains this web setting. Merge it into the existing JSON; do not replace the entire file if it has other properties:

```
{
  "expo": {
    "web": {
      "bundler": "metro"
    }
  }
}
```

This is only the relevant snippet. If an expo object already exists, add web inside it and preserve other settings. JSON does not allow comments or trailing commas.

5F. Test NativeWind — FILE: App.js

```
import './global.css';
import { View, Text } from 'react-native';

export default function App() {
  return (
    <View className="flex-1 items-center justify-center bg-indigo-600">
      <Text className="text-4xl font-bold text-white">
        Hello, Tailwind!
      </Text>
    </View>
  );
}
```

```
npx expo start -c
```

-c clears Metro's cache after configuration changes. Expected: bold white text on an indigo background. If styling does not appear, check the global.css import, config filenames, content paths, and installed versions.

6. Part E — Build the screen with components

Create a folder named components beside App.js. Inside it create TopBar.js and NavBar.js. Filenames and capitalization must match the imports.

6A. FILE: components/TopBar.js

```
import { View, Text } from 'react-native';

export default function TopBar() {
  return (
    <View className="flex-row items-center justify-between bg-indigo-600 px-4 py-4">
      <Text className="text-xl font-bold text-white">My First App</Text>
      <Text className="text-2xl text-white">■</Text>
    </View>
  );
}
```

flex-row places children side by side; items-center aligns them vertically; justify-between pushes title and menu apart; bg-indigo-600 sets background; px-4 and py-4 add horizontal/vertical padding. Put text styling on Text, not View.

6B. FILE: components/NavBar.js

```
import { View, Text, Pressable } from 'react-native';

export default function NavBar() {
  return (
    <View className="flex-row border-t border-slate-200 bg-white">
      <Pressable className="flex-1 items-center py-2">
        <Text className="text-lg">■</Text>
        <Text className="text-sm font-bold text-indigo-600">Home</Text>
      </Pressable>
      <Pressable className="flex-1 items-center py-2">
        <Text className="text-lg">■</Text>
        <Text className="text-sm text-slate-500">Search</Text>
      </Pressable>
      <Pressable className="flex-1 items-center py-2">
        <Text className="text-lg">■</Text>
        <Text className="text-sm text-slate-500">Profile</Text>
      </Pressable>
    </View>
  );
}
```

Pressable is a customizable pressable container. No onPress handler is added because the lab is static. flex-1 makes items share width; each item contains two Text components stacked vertically. Home is visually selected by its bold indigo label.

6C. FILE: App.js — assemble the screen

```
import './global.css';

import { Text, Pressable, ScrollView } from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import TopBar from './components/TopBar';
import NavBar from './components/NavBar';

export default function App() {
  return (
    <SafeAreaProvider>
      <SafeAreaView className="flex-1 bg-slate-50">
        <TopBar />
        <ScrollView className="flex-1" contentContainerClassName="p-4">
          <Text className="text-3xl font-bold text-slate-800">
            Hello, World!
          </Text>
          <Text className="mt-2 text-base text-slate-600">
            Welcome to React Native. This is my first mobile
            application built with Expo and NativeWind.
          </Text>
          <Pressable className="mt-6 items-center rounded-xl bg-indigo-600 py-3">
            <Text className="font-bold text-white">Get Started</Text>
          </Pressable>
          <Pressable className="mt-3 items-center rounded-xl border border-indigo-600 py-3">
            <Text className="font-bold text-indigo-600">Learn More</Text>
          </Pressable>
        </ScrollView>
        <NavBar />
      </SafeAreaView>
    </SafeAreaProvider>
  );
}
```

App.js imports global.css once, imports each React Native component used in this file, and imports the two local components. SafeAreaProvider supplies safe-area context; SafeAreaView creates the screen. TopBar comes first, ScrollView fills the middle, and NavBar is below the scrollable area. contentContainerClassName adds padding to the ScrollView's inner content.

Every visible text string must be inside Text, including button labels. Do not add onPress handlers in this lab; the buttons and navigation items are expected to be nonfunctional.

7. Part F — All seven customization tasks

1. Change the app title in components/TopBar.js, e.g. Gemini Space.

2. Change the top bar background and primary button background from bg-indigo-600 to another color such as bg-sky-600. Optionally match the secondary button border/text color.

3. Replace the paragraph in App.js with your name, course, and one hobby.

4. Add a third Delete button after Learn More and before the ScrollView closes.

5. Add a fourth Settings item in components/NavBar.js before the outer View closes.

6. Add text-center to both the heading and paragraph in App.js.

7. Make a different navigation item look selected, e.g. Profile bold and sky-blue; Home gray.

7A. Task 4 — insert in App.js after Learn More

```
<Pressable className="mt-3 items-center rounded-xl bg-red-500 py-3">
  <Text className="font-bold text-white">Delete</Text>
</Pressable>
```

7B. Task 5 — insert in components/NavBar.js before the outer </View>

```
<Pressable className="flex-1 items-center py-2">
  <Text className="text-lg">■■</Text>
  <Text className="text-sm text-slate-500">Settings</Text>
</Pressable>
```

7C. Task 6 — centered heading and personal paragraph (App.js)

```
<Text className="text-3xl font-bold text-slate-800 text-center">
  Hello, World!
</Text>
<Text className="mt-2 text-base text-slate-600 text-center">
  Hi! I'm [Your Name], a [Your Course] student.
  I enjoy [Your Hobby] in my free time.
</Text>
```

7D. Task 7 — make Profile selected (components/NavBar.js)

```
<Text className="text-sm font-bold text-sky-600">Profile</Text>
```

Remove the bold/color classes from Home and make Home gray (text-slate-500). Replace placeholders before submission. Delete is only a visual button; it must not delete anything.

8. Complete final customized code

These full files combine all seven tasks. Replace the name/course/hobby placeholders.

Final FILE: components/TopBar.js

```
import { View, Text } from 'react-native';

export default function TopBar() {
  return (
    <View className="flex-row items-center justify-between bg-sky-600 px-4 py-4">
      <Text className="text-xl font-bold text-white">Gemini Space</Text>
      <Text className="text-2xl text-white">■</Text>
    </View>
  );
}
```

Final FILE: components/NavBar.js

```
import { View, Text, Pressable } from 'react-native';

export default function NavBar() {
  return (
    <View className="flex-row border-t border-slate-200 bg-white">
      <Pressable className="flex-1 items-center py-2">
        <Text className="text-lg">■</Text>
        <Text className="text-sm text-slate-500">Home</Text>
      </Pressable>
      <Pressable className="flex-1 items-center py-2">
        <Text className="text-lg">■</Text>
        <Text className="text-sm text-slate-500">Search</Text>
      </Pressable>
      <Pressable className="flex-1 items-center py-2">
        <Text className="text-lg">■</Text>
        <Text className="text-sm font-bold text-sky-600">Profile</Text>
      </Pressable>
      <Pressable className="flex-1 items-center py-2">
        <Text className="text-lg">■■</Text>
        <Text className="text-sm text-slate-500">Settings</Text>
      </Pressable>
    </View>
  );
}
```

Final FILE: App.js

```
import './global.css';

import { Text, Pressable, ScrollView } from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import TopBar from './components/TopBar';
import NavBar from './components/NavBar';

export default function App() {
  return (
    <SafeAreaProvider>
      <SafeAreaView className="flex-1 bg-slate-50">
        <TopBar />
        <ScrollView className="flex-1" contentContainerClassName="p-4">
          <Text className="text-3xl font-bold text-slate-800 text-center">
            Hello, World!
          </Text>
          <Text className="mt-2 text-base text-slate-600 text-center">
            Hi! I'm [Your Name], a [Your Course] student.
            I enjoy [Your Hobby] in my free time.
          </Text>
          <Pressable className="mt-6 items-center rounded-xl bg-sky-600 py-3">
            <Text className="font-bold text-white">Get Started</Text>
          </Pressable>
          <Pressable className="mt-3 items-center rounded-xl border border-sky-600 py-3">
            <Text className="font-bold text-sky-600">Learn More</Text>
          </Pressable>
          <Pressable className="mt-3 items-center rounded-xl bg-red-500 py-3">
            <Text className="font-bold text-white">Delete</Text>
          </Pressable>
        </ScrollView>
        <NavBar />
      </SafeAreaView>
    </SafeAreaProvider>
  );
}
```

9. Troubleshooting quick reference

Problem	Fix
node or npx not recognized	Reopen terminal; check Node.js installation and PATH.
adb not recognized	Check SDK path, ANDROID_HOME and %ANDROID_HOME%\platform-tools in user Path; reopen terminal.
No Android device found	Start emulator and wait for home screen; run adb devices.
Tailwind classes do nothing	Check import './global.css' in App.js, content patterns and NativeWind configs; run npx expo start -c.
Cannot find module nativewind/babel	Re-run install commands from the project root; verify NativeWind installed.
Reanimated / Worklets error	Run npx expo install react-native-worklets, then npx expo install --fix, then restart with -c.
Text strings must be rendered within Text	Wrap plain strings in <Text>.
View or Text missing / undefined	Import it from react-native in the file where used.
Element type is invalid	Check export default, import path, filename and capitalization.
Changes do not appear	Save, press r in Expo terminal, or restart with npx expo start -c.
Web preview blank	Check app.json expo.web.bundler is metro; restart with -c.
Emulator slow or won't start	Check virtualization; enable Windows Hypervisor Platform if needed; close heavy programs or use lighter device.
PowerShell blocks scripts	Try Command Prompt, as the lab suggests.

10. Tailwind class cheat sheet

Class	Meaning
flex-1	Fill available flex space.
flex-row	Arrange children horizontally.
items-center	Align children on the cross-axis.
justify-center	Center along main axis.
justify-between	Put space between children.
p-4 / px-4 / py-2	Padding all sides / horizontal / vertical.
mt-4	Top margin.
bg-white / bg-slate-50 / bg-sky-600	Background colors.
text-sm / text-base / text-xl / text-3xl	Text sizes.
font-bold / font-semibold	Text weight.
text-white / text-slate-600	Text colors.
rounded / rounded-xl / rounded-full	Corner rounding.
border / border-t / border-slate-200	Border styles.
text-center	Center text alignment.
active:opacity-80	Optional pressed-state opacity effect.

11. Questionnaire answers

Part C Q1. What happens if plain text is placed directly inside View without Text?

React Native shows an error because text strings must be rendered inside Text. View is a layout container.

Part C Q2. What does flex: 1 or flex-1 do?

It lets a component expand to fill available space in its parent flex layout.

Section 8 Q3. React Native equivalents of div, p, img and button?

div → View; p → Text; img → Image; button → Pressable or Button.

Section 8 Q4. Why do global.css and tailwind.config.js exist?

global.css is the Tailwind stylesheet entry point; tailwind.config.js tells Tailwind which files to scan and loads the NativeWind preset.

Section 8 Q5. Why use Pressable instead of Button for custom styling?

Pressable is flexible and can be styled with classes; the built-in Button has limited styling options.

Section 8 Q6. flex-row vs View's default direction?

flex-row arranges children horizontally; View defaults to a column layout, stacking children vertically.

Section 8 Q7. What makes a button respond when pressed?

An onPress handler defines the action. The current lab intentionally does not add one.

Section 8 Q8. Why import View and Text, and from where?

They are React Native components, not built-in HTML tags; import them from react-native.

Section 8 Q9. Why separate TopBar and NavBar, and what enables their use in App.js?

Separate files make code organized and reusable. export default exposes each component; import brings it into App.js.

12. Grading rubric

Criterion	Points
Environment verified and project created	15
NativeWind and Tailwind correctly configured	20
Static screen with top bar, text, buttons and bottom bar	30
All seven customization tasks	20
Separate TopBar and NavBar, correct imports, readable code	5
Answers to questions in Section 8	10
TOTAL	100

13. Final checklist

■ Node.js and npm work in a new terminal.

■ Android SDK path and adb work; emulator boots to Android home screen.

■ Expo SDK 57 project created and opens in VS Code.

■ Hello World worked before NativeWind setup.

■ Installed versions follow lab: NativeWind 4.2.7 and Tailwind CSS 3.4.

■ tailwind.config.js, global.css, babel.config.js, metro.config.js and app.json are configured.

■ Tailwind test screen displays white text on indigo.

■ components/TopBar.js and components/NavBar.js exist and are imported in App.js.

■ Final screen shows top bar, centered heading/paragraph, three buttons and four bottom items.

■ All seven customization tasks are complete; personal placeholders are replaced.

■ Buttons have no onPress handlers for this lab.

■ Questionnaire answers are ready for Google Classroom.

Submission filename in the manual: Surname_ReactNativeQuestionnare (example: Santos_ReactNativeQuestionnare). The manual spells “Questionnare” that way; follow any clarification from your instructor.

Source basis: uploaded “Laboratory 1: Getting Started with React Native (Expo + Tailwind) - Updated” PDF. The D: drive notes are supplemental; the core lab sequence, version targets, files, tasks, troubleshooting, rubric and questions follow the manual.